CS 483, Fall 2026

Programming Assignment 2
Harris Corner Detection

Due Date: Oct 22 Thu, 23:55 (HST)

In this assignment, you will implement the main steps of the Harris corner detector and use it to find interest points in an image. A corner is an image location whose appearance changes strongly when a small window is shifted in more than one direction. These points are useful for many computer vision tasks, including image matching, tracking, and geometric estimation.

Unlike Programming Assignment 1, you do **not** need to implement image convolution yourself. You may use OpenCV or SciPy functions for image filtering, Gaussian smoothing, and local maximum filtering. However, you must implement the Harris response from the image derivatives yourself. You may not call `cv2.cornerHarris`, `cv2.goodFeaturesToTrack`, or another built-in corner detector to produce the required result.

1 Instructions

1. **Integrity and collaboration:** Students are encouraged to work in groups but each student must write their own code and write-up. If you work as a group, include the names of your collaborators in your write up. Code should **NOT** be shared or copied. Please **DO NOT** use external code or large-language models such as ChatGPT to vibe code. If you use LLMs, please attach and report on how you used LLMs to finish the assignment. Plagiarism is strongly prohibited and may lead to failure of this course.

2. **Start early!** Especially those not familiar with Python.

3. **Questions:** If you have any questions, please look at Discord first. Other students may have encountered the same problem, and it may be solved already.

4. **Write-up:** Your write-up should mainly consist of three parts: (i) your answers to theory questions, (ii) the resulting images from the intermediate steps of your Harris detector, and (iii) the discussion for the experiment. Please note that we **DO NOT** accept handwritten scans for your write-up in this assignment. Please type your answers to theory questions and discussions for experiments electronically.

5. **Submission:** Your submission for this assignment should be a zip file, `<UHID.zip>`, composed of your write-up and your Python implementations. Please make sure to remove temporary files you generated.

   Your final upload **MUST** have the files arranged and named in this layout:

- <UHID>.zip
  - <UHID>.pdf
  - python/
    - harrisScript.py
    - myImageGradient.py
    - myHarrisResponse.py
    - myHarrisCorners.py
    - any helper functions you need

Note that assignments that do not conform to the guidelines will be deducted 10 points.

6. You should submit reasonably vectorized code, i.e. avoid looping over individual image pixels when NumPy/OpenCV operations can perform the same computation efficiently.

7. **File paths:** Please make sure that any file paths that you use are relative and not absolute. For example, use `cv2.imread('images/hawaii_hall.jpg')` rather than an absolute path on your own computer.

2 Theory questions

Type down your answers for the following questions in your write-up. Each question should only take a couple of lines. You do not need lengthy calculations.

Q2.1 Harris Corner Detector (20 points)

1. Why does a corner produce large intensity changes in more than one direction, while an edge usually produces a large change mainly in one direction?
2. The second-moment matrix at a pixel is

$$M = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}.$$

   In terms of the two eigenvalues of M, describe a flat region, an edge, and a corner.
3. Why do we average the gradient products over a neighborhood instead of using I_x and I_y at only one pixel?
4. What is the purpose of non-maximum suppression when selecting corners from the Harris response map?

3 Implementation

We have included a main script named `harrisScript.py` that takes care of reading the image, making function calls to the different steps of the Harris detector, and generating images showing the output and intermediate results. You are free to modify the script as you want, but note that TAs may run the original `harrisScript.py` while grading. Please make sure your functions run correctly with the original script.

Every script and function you write in this section should be included in the `python/` directory. Please include the requested resulting images in your write-up.

Q3.1 Image gradients (20 points)

Write a function that computes the horizontal and vertical image derivatives:

```
[imgx, imgy] = myImageGradient(img0)
```

As input, the function takes a grayscale image `img0`. The outputs `imgx` and `imgy` should have the same size as the input image. Convert the image to floating point before filtering.

Use the following derivative kernels:

$$K_x = \frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_y = K_x^T.$$

You may use `cv2.filter2D`, `scipy.ndimage.convolve/correlate`, or an equivalent image filtering function. You do **not** need to implement convolution yourself. Display `imgx` and `imgy` for the required test image in your write-up.

Q3.2 Harris response (25 points)

Write a function that calculates the Harris corner response from the image derivatives:

```
[R, Sxx, Syy, Sxy] = myHarrisResponse(imgx, imgy, sigma, k)
```

First compute the three gradient products

$$I_x^2, \qquad I_y^2, \qquad I_x I_y.$$

Smooth each product with a Gaussian filter to obtain

$$S_{xx} = G_\sigma * I_x^2, \qquad S_{yy} = G_\sigma * I_y^2, \qquad S_{xy} = G_\sigma * I_x I_y.$$

You may use `cv2.GaussianBlur`, `scipy.ndimage.gaussian_filter`, or another equivalent Gaussian filter.

At every pixel, these values form the second-moment matrix

$$M = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}.$$

Compute the Harris response

$$R = \det(M) - k\,\mathrm{trace}(M)^2 = S_{xx}S_{yy} - S_{xy}^2 - k(S_{xx} + S_{yy})^2.$$

Use $k = 0.04$ as the default value. Your code may not call `cv2.cornerHarris` or another built-in corner detector. **Include intermediate result images at each step.**

Q3.3 Finding corners (20 points)

Write a function that finds strong, spatially separated corners in the Harris response map:

```
corners = myHarrisCorners(R, thresholdRel, nmsSize)
```

First discard weak responses using a relative threshold, for example

$$R(x, y) > \texttt{thresholdRel}\ \max(R).$$

Then apply non-maximum suppression so that several neighboring pixels around the same physical corner are not all returned as separate detections. You may use `scipy.ndimage.maximum_filter`, OpenCV's `dilate`, or your own implementation. Return the detected corner locations as an $N \times 2$ array of $[x, y]$ coordinates.

4 Experiments

Q4.1 Write-up (15 points)

Test your implemented Harris detector on the Hawai'i Hall image shown in Figure 1. The image should be saved as `python/images/hawaii_hall.jpg`. The exact image is available from the link in the figure caption and can also be downloaded using the provided `downloadExampleImage.py` script.

Use the following parameters as a starting point:

$$\sigma = 1.5, \qquad k = 0.04, \qquad \texttt{thresholdRel} = 0.01, \qquad \texttt{nmsSize} = 7.$$

You may adjust the threshold or NMS size if necessary to obtain a reasonable result.

Include the following in your write-up:

1. the input image;
2. the horizontal and vertical derivatives, I_x and I_y;
3. the Harris response map R;
4. the final image with the detected corners marked;
5. a short discussion of where the detector works well and where it produces questionable or missing corners; and

6. change one parameter (for example σ or `thresholdRel`), rerun the detector, and briefly explain the effect.



Figure 1: Required test image: Hawai‘i Hall. Photo by Sdkb, Wikimedia Commons, CC BY-SA 4.0. Image link.

Figure 2 shows an example of the expected final visualization. Your corner locations do not need to match this illustration exactly; the number and location of detected corners depend on the image and parameter choices.

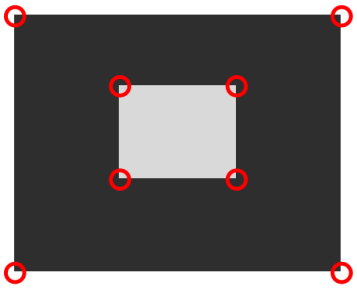


Figure 2: Example Harris corner detection output. Detected corner locations are marked on the input image.

5 Non-maximum suppression

Non-maximum suppression (NMS) is used to keep local maxima and suppress nearby responses. For each pixel in the Harris response map, compare its value with the values in a local neighborhood (for example, a 7×7 window). Keep the response only if it is the maximum in that neighborhood and is above the response threshold. This prevents many adjacent pixels around one physical corner from being reported as separate corners.

In this assignment, you do not need to implement the local maximum filter from scratch. The SciPy function `scipy.ndimage.maximum_filter` or OpenCV's `dilate` can be used to identify local maxima.